DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semanas 16 a 18
Checkpoint final, banca e revisão para a 3ª VA

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Esta apostila acompanha a reta final do semestre. Nas semanas 16 a 18 a sua API passa por três provas: o checklist dos critérios de aceite do Mapa de Construção, a bancada de testes rodando ao vivo e a banca, que pergunta por que cada decisão foi tomada. A teoria desta apostila trata do que sustenta essa conversa: como defender uma escolha técnica, como argumentar um trade-off, como apresentar uma arquitetura com o modelo C4, como declarar dívida técnica e limitações com honestidade e como uma API de qualidade é avaliada. No fim, uma síntese do semestre e os pontos de revisão para a 3ª VA, na semana 19.

Conteúdo

1. Objetivos da semana
2. O que é uma defesa técnica
3. Trade-offs: como argumentar
4. Apresentar arquitetura: o modelo C4
5. Dívida técnica e limitações
6. Rubrica de qualidade de uma API
7. Como a bancada valida o contrato
8. O checkpoint final no ServiceHub
9. A banca: roteiro e perguntas
10. Síntese do semestre e revisão para a 3ª VA
11. Construindo com IA
12. Glossário
13. Exercícios
14. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final das semanas 16 a 18 você deve ser capaz de:

1. conduzir uma defesa técnica, explicando uma decisão pelo problema, pelas alternativas, pelo critério, pelo custo e pela evidência;
2. desenhar e apresentar a arquitetura do ServiceHub nos níveis de contexto, container e componente do modelo C4;
3. distinguir dívida técnica de defeito e declarar as limitações reais do projeto com motivo, impacto e caminho de solução;
4. autoavaliar a API com uma rubrica de qualidade e conferir, item a item, os critérios de aceite da seção 12 do Mapa;
5. explicar o que cada um dos 15 passos da bancada de testes verifica e diagnosticar um passo vermelho a partir do motivo informado.

CALENDÁRIO DA RETA FINAL

A banca acontece entre as semanas 16 e 18. Data, horário e ordem das equipes são divulgados no AVA, conforme o calendário de eventos da instituição. A versão que vai à banca é a marcada com a tag `etapa-final` no repositório da equipe. A 3ª VA vem depois, na semana 19, e cobra os conceitos do semestre, não só o projeto.

2. O que é uma defesa técnica

↑ conteúdo

Uma **defesa técnica** é a explicação de uma escolha de engenharia diante de quem pode questioná-la. Ela não tenta provar que a solução é perfeita, porque nenhuma é. Ela mostra que a solução foi escolhida *de propósito*: a equipe sabia qual problema estava resolvendo, considerou outras opções, usou um critério claro e sabe o que pagou pela escolha.

Código que funciona pode ter sido copiado, gerado ou montado por tentativa e erro; a única forma de saber se a equipe domina o que entregou é perguntar *por quê*.

ANALOGIA

Um médico que explica um tratamento não diz apenas "tome este remédio". Ele diz qual é o diagnóstico, quais opções existiam, por que escolheu esta e quais riscos ela traz. O paciente pode até preferir outra opção, mas sabe que a escolha não foi aleatória. A defesa técnica tem a mesma estrutura.

Descrever não é defender

A confusão mais comum na banca é responder com uma descrição. Descrever é dizer *o que* foi feito. Defender é dizer *por que*, *em vez de quê* e *a que custo*.

Descrição

"Usamos JWT para autenticação."

Correto, mas não mostra se a equipe entende a escolha. Qualquer pessoa que leu o `pom.xml` diria o mesmo.

Defesa

"Usamos JWT em vez de sessão no servidor porque a API é sem estado. O custo é que um token emitido não pode ser revogado facilmente. Para reduzir isso, o usuário do token é conferido no banco a cada requisição: desativado, recebe 401."

As cinco partes de uma boa resposta

Parte	Pergunta que responde	Exemplo genérico (app de agendamento)
Problema	Que força obrigou a decidir?	Dois clientes podem reservar o mesmo horário ao mesmo tempo.
Alternativas	O que mais foi considerado?	Verificar no código antes de gravar, ou uma restrição de unicidade no banco.
Critério	O que pesou mais?	Garantia sob concorrência.
Custo	O que se perdeu?	Tratar a exceção do banco e traduzi-la num erro claro.
Evidência	Como você sabe que funciona?	Um teste que tenta duas reservas iguais e recebe 409 na segunda.

No ServiceHub essa estrutura já existe por escrito: são os **ADRs** (*Architecture Decision Records*, registros de decisão de arquitetura) da pasta `docs/adr/`, no formato proposto por Michael Nygard, com contexto, decisão e consequências. O Mapa pede ao menos três; na banca, o ADR é o roteiro da resposta.

3. Trade-offs: como argumentar

↑ conteúdo

Trade-off é uma troca: ganhar uma característica abrindo mão de outra. Richards e Ford resumem a arquitetura de software numa frase conhecida, a de que tudo em arquitetura é um trade-off. A consequência prática é direta: se uma escolha parece ter só vantagens, a análise ainda não terminou.

O argumento segue os mesmos cinco passos da defesa técnica (seção 2): contexto, opções, critério, custo e evidência. Sem o custo, é propaganda; sem a evidência, é opinião.

Um exemplo fora do ServiceHub

O catálogo de uma loja virtual recebe muito mais leituras do que alterações (contexto). A equipe pode consultar o banco a cada visita ou guardar o catálogo em cache por alguns minutos (opções). A velocidade de leitura pesou mais (critério), então escolheu o cache. Em troca, um preço alterado pode levar alguns minutos para aparecer (custo). Um teste de carga comparando as duas versões mostra a diferença de tempo de resposta (evidência).

O argumento não diz que cache é "melhor": diz que, *naquele contexto* e com *aquele critério*, ele venceu. Para o saldo de uma conta bancária, a consistência pesaria mais, e a escolha seria outra.

O mesmo raciocínio no ServiceHub

O ADR-0005 decide entre H2 e Testcontainers nos testes. **Contexto:** as migrações Flyway usam SQL que só o PostgreSQL entende, como `BIGSERIAL` e `CHECK (status IN (...))`. **Opções:** H2 em memória, sem dependências; ou PostgreSQL 16 real num container. **Critério:** fidelidade ao banco de produção. **Custo:** Docker obrigatório para rodar `mvnw verify` e alguns segundos a mais por execução. **Evidência:** `mvnw verify` aplica as migrações reais e roda os testes de integração contra elas, inclusive no GitHub Actions.

CONFUSÃO COMUM

"Porque é a melhor prática" não é critério, é apelo à autoridade. A banca vai perguntar: melhor para quê, em que contexto, comparada com o quê? Outra armadilha é listar só vantagens. Quem não sabe o custo da própria decisão provavelmente não decidiu: aceitou o que veio pronto, às vezes da IA.

4. Apresentar arquitetura: o modelo C4

↑ conteúdo

Na banca você vai precisar mostrar como o sistema é organizado em poucos minutos. Um diagrama sem convenção vira um amontoado de caixas e setas que cada pessoa lê de um jeito. O **modelo C4**, proposto por Simon Brown, resolve isso com uma ideia simples: descrever o software em **níveis de zoom**, cada um para um público, como um mapa digital que vai do país à rua. Para a banca, contexto e container quase sempre bastam. Richards e Ford o apresentam ao lado de UML e ArchiMate como forma de diagramar arquitetura.

Nível	O que mostra	Para quem
1. Contexto	O sistema como uma caixa só, as pessoas que o usam e os sistemas externos com que conversa.	Qualquer público, inclusive não técnico.

Nível	O que mostra	Para quem
2. Container	As partes que rodam separadamente (aplicações, bancos, filas) e como se comunicam.	Equipe técnica e banca.
3. Componente	Os blocos principais dentro de um container e a responsabilidade de cada um.	Quem vai mexer no código.
4. Código	Classes e relações. Raramente desenhado à mão: a IDE já mostra.	Casos pontuais.

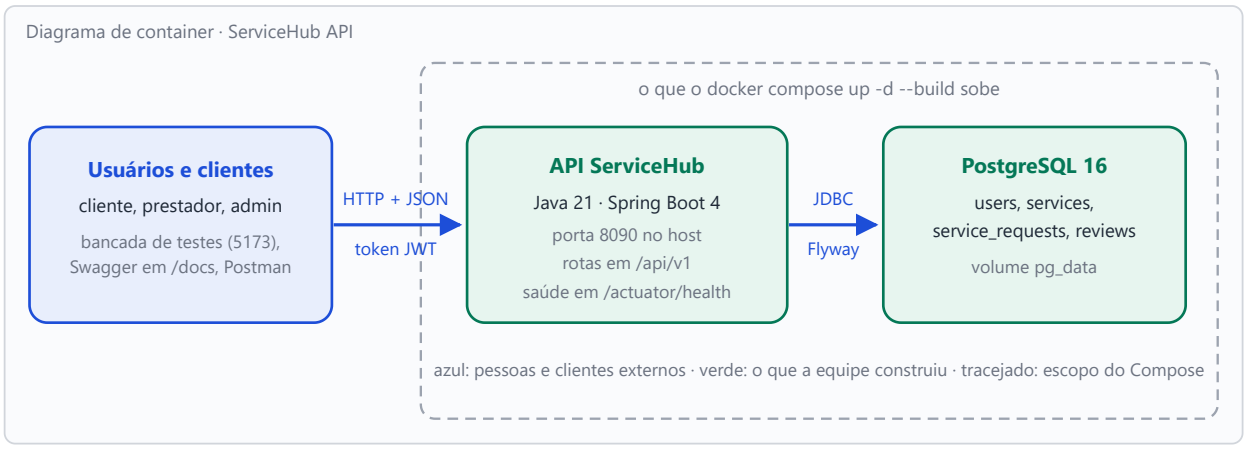
Um exemplo genérico: biblioteca universitária

No **contexto**, há uma caixa "Sistema da biblioteca", as pessoas "aluno" e "bibliotecário" e o sistema externo "cadastro acadêmico", que informa quem está matriculado. No nível de **container**, a caixa se abre em um aplicativo web, uma API e um banco de dados, com setas dizendo "HTTPS + JSON" e "SQL". No nível de **componente**, a API se abre em empréstimos, multas e catálogo.

CONFUSÃO COMUM

"Container" no C4 não significa container Docker. É qualquer coisa que roda e guarda estado separadamente: uma aplicação, um banco, um app de celular. Às vezes coincide com um container Docker, como no ServiceHub, mas não é a mesma ideia.

O ServiceHub no nível de container



Nível 2 do C4 para o ServiceHub. Cada seta diz o que trafega; a legenda explica cores e molduras.

O pgAdmin, que o Compose do projeto sobe só no perfil `tools`, fica fora do diagrama principal: é ferramenta de apoio, não parte do sistema. O diagrama mostra o que é *necessário* para o sistema funcionar.

O zoom seguinte: componentes da API



Nível 3 do C4: a caixa verde da API aberta nos blocos que a equipe programou.

Quatro regras para apresentar

- **O todo antes da parte.** Mostre onde a peça se encaixa antes de dar zoom nela.
- **Título e legenda.** Todo diagrama diz o que mostra e o que cada forma, cor e linha significa. Formas iguais para coisas diferentes, como a mesma caixa para banco e para fila, confundem.
- **Setas com rótulo.** Uma seta sem rótulo não diz nada. Escreva o que trafega e em que sentido.
- **O slide é metade.** O slide mostra; quem apresenta explica. Richards e Ford recomendam revelar o diagrama aos poucos, na ordem da fala.

5. Dívida técnica e limitações

↑ conteúdo

Dívida técnica é o custo futuro de um atalho tomado hoje. A metáfora é de Ward Cunningham: como num empréstimo, o atalho dá velocidade agora, mas cobra **juros**. Enquanto a dívida não é paga, com refatoração, cada mudança naquele trecho sai mais cara.

Exemplo genérico de juros: uma regra de desconto foi copiada em cinco lugares para entregar rápido. No dia em que a regra mudar, será preciso alterar os cinco trechos e lembrar de todos. Esquecer um gera um defeito que ninguém percebe até o cliente reclamar.

O quadrante de Fowler

Martin Fowler classificou a dívida em dois eixos: se foi tomada de propósito (deliberada) ou sem perceber (inadvertida), e se foi uma decisão sensata (prudente) ou descuidada (imprudente).

	Imprudente	Prudente
Deliberada	"Não temos tempo para testes."	"Entregamos agora, anotamos o atalho e pagamos na próxima etapa."
Inadvertida	"Camadas? O controller fala direto com o banco."	"Só agora entendemos como deveríamos ter feito."

Dívida não é pecado; o problema é a dívida *escondida*. A dívida deliberada e prudente é registrada, com motivo e plano de pagamento, num ADR, no README ou numa issue. Código gerado por IA e aceito sem revisão costuma cair no quadrante inadvertido e imprudente: ninguém decidiu nada, apenas aceitou.

DÍVIDA NÃO É DEFEITO

Um defeito é o sistema fazendo algo errado, como devolver 500 quando deveria devolver 409. Dívida é o sistema fazendo o certo de um jeito que encarece as mudanças. Defeito se corrige antes da banca; dívida se declara.

Declarar limitações com honestidade

Uma **limitação** é algo que o sistema conscientemente não faz. Uma limitação declarada mostra domínio; uma descoberta pela banca mostra descuido. A forma de declarar tem sempre quatro colunas: o quê, por quê, impacto e caminho.

O projeto de referência do ServiceHub tem estas limitações reais:

Limitação	Por que ficou assim	Impacto	Como resolveríamos
Sem limite de taxa (*rate limiting*)	Fora do escopo do semestre.	O login fica exposto a tentativas em massa de senha (força bruta).	Limite de requisições por cliente num gateway ou num filtro, respondendo 429.
Sem rastreamento distribuído (*tracing*)	Há uma única API; o `X-Request-Id` já liga a requisição às linhas de log.	Não seria possível seguir uma chamada através de outros serviços.	Tracing com identificadores de *trace* e *span*, se surgirem outros serviços.
Só a v1, sem headers de depreciação	Ainda não houve mudança incompatível no contrato.	Nenhum hoje; vira risco quando nascer a v2.	Filtro que emite os headers `Deprecation` e `Sunset` nas rotas da v1.
Sem alertas automáticos	O painel do Grafana basta no ambiente da disciplina.	Alguém precisa olhar o painel para notar o problema.	Regras de alerta no Prometheus ligadas ao SLO.

PARA LEMBRAR

"Não deu tempo" é desculpa, não limitação. A coluna do caminho é o que transforma a frase em engenharia: mostra que a equipe sabe como resolveria e o que isso custaria.

6. Rubrica de qualidade de uma API

↑ conteúdo

Uma **rubrica** é uma tabela de critérios com níveis de desempenho descritos, usada para avaliar algo de forma consistente. Ela troca a impressão geral por perguntas com resposta verificável.

Critério	A pergunta	Evidência que responde
Consistência	Toda rota segue as mesmas convenções?	Mesmos nomes, mesmos status e ProblemDetail em 400, 401, 403, 404 e 409.
Segurança	Só quem pode faz, e nada sensível vaza?	401 sem token, 403 na ordem alheia, nenhuma senha ou hash na resposta.
Documentação	Alguém de fora usa e entende sem o autor?	Swagger em `/docs` coerente com as respostas reais; README e ADRs.
Testabilidade	As regras estão protegidas por testes?	`mvnw verify` verde; quebrar uma regra deixa um teste vermelho.
Operabilidade	Sobe, se diagnostica e se observa sem heroísmo?	`docker compose up`, health UP, `X-Request-Id`, logs e métricas.

Cada critério tem três níveis: **ausente**, **parcial** ou **atendido com evidência**. Só o terceiro se defende na banca. "Temos testes" é parcial; "temos um teste por regra, e aqui está o que falha se eu quebrar a RN09" é atendido com evidência.

A rubrica serve de autoavaliação; os critérios oficiais estão na seção 12 do Mapa (seção 8 desta apostila). Onde der "parcial", corrija até a banca ou declare a limitação.

7. Como a bancada valida o contrato

↑ conteúdo

A **bancada de testes** é uma ferramenta que roda a mesma sequência de chamadas contra a API de cada equipe e confere, passo a passo, se as respostas cumprem o contrato do Mapa. Ela é um teste de **caixa-preta**: não lê o seu código, só fala HTTP com a API, como faria qualquer cliente real. Por isso ela é justa: a mesma ferramenta, com as mesmas verificações, para todas as equipes.

Fora da disciplina, a ideia se chama suíte de conformidade: o dono de um contrato a roda contra cada implementação. A bancada fica na pasta `frontend/` do projeto de referência. É HTML, CSS e JavaScript puro, sem etapa de build; o Playwright apenas dirige um navegador por ela e transforma cada passo numa etapa do relatório.

Os 15 passos

#	Passo	Chamada	O que confere
1	API no ar	`GET /api/v1/info`	200 e `status` igual a "OK"

#	Passo	Chamada	O que confere
2–3	Cadastrar prestador e cliente	`POST /api/v1/auth/register`	201, `id`, papel correto, nenhuma senha na resposta
4–5	Login de cada um	`POST /api/v1/auth/login`	200 e `accessToken`
6	Prestador publica serviço	`POST /api/v1/services`	201 e `providerId` igual ao do token
7	Cliente contrata	`POST …/service-requests`	201, PENDING, `totalPrice` copiado
8–10	Prestador aceita, inicia, conclui	`PATCH …/{id}/status`	200 e status ACCEPTED, IN_PROGRESS, COMPLETED
11	Cliente avalia	`POST …/{id}/reviews`	201 e nota 5
12	Avaliações públicas	`GET …/services/{id}/reviews`	200, página com a avaliação em `content`
13	Sem token não cria serviço	`POST /api/v1/services`	401 em ProblemDetail, `type` … `unauthorized`
14	Cliente não publica serviço	`POST /api/v1/services`	403 em ProblemDetail, `type` … `forbidden`
15	Ordem concluída não volta a PENDING	`PATCH …/{id}/status`	409 em ProblemDetail, `type` … `invalid-transition`

Os passos 2 a 12 são o fluxo completo da seção 12 do Mapa; os três últimos, o contrato de erro.

Como um passo é escrito

Cada passo do `frontend/app.js` declara método, rota, corpo, status esperado e uma função de verificação que devolve `null` quando passa ou um texto explicando o que faltou. O passo 7, por exemplo (resumido):

```
id: 'create-request', method: 'POST', path: () => '/api/v1/service-requests',
expect: 201, needs: ['clientToken', 'serviceId'],
check: (b) => requireField(b, 'id') || expectStatus(b, 'PENDING') ||
  (Number(b.totalPrice) === 150 ? null
    : 'totalPrice deveria copiar o preço do serviço (150.00)'),
```

O campo `needs` explica por que um vermelho arrasta outros: se o passo 6 falhar, não há `serviceId`, e os passos seguintes aparecem como pulados. A regra prática é sempre **corrigir o primeiro vermelho**. Nos passos 13 a 15, a função `problemType` confere primeiro o cabeçalho `Content-Type` (precisa ser `application/problem+json`) e depois se o campo `type` termina no sufixo esperado.

Rodando a bancada

Pré-requisitos: Node 18 ou superior, a API da equipe de pé e CORS liberado para `http://localhost:5173`. No PowerShell, dentro da pasta `frontend`:

```
npm install                      # e, se preciso: npx playwright install chromium
$env:API_URL = "http://localhost:8090"; $env:EQUIPE = "Grupo 3"
npm run aovivo      # navegador visível, ritmo de apresentação
npm run teste       # sem janela, rápido: para conferir antes
npm run relatorio   # relatório HTML com os anexos das falhas
```

No Git Bash ou Linux a sintaxe das variáveis muda: `API_URL=http://localhost:8090 EQUIPE="Grupo 3" npm run aovivo`. Cada execução cria prestador e cliente novos, com e-mails únicos baseados na hora, então o ciclo pode ser repetido contra o mesmo banco. Há ainda `npm run bancada`, que abre a tela em `localhost:5173` para uso manual; nela, o botão **Copiar relatório** gera um resumo em texto para colar no AVA.

Quando um passo falha

Sintoma na bancada	Causa provável
"Sem resposta" em todos os passos	CORS sem `http://localhost:5173`, ou a API fora do ar.
Passo 1 falha	API em outra porta, ou `/api/v1/info` sem `status` "OK".
Cadastro: esperado 201, recebido 400	DTO com nomes diferentes do Mapa: `fullName`, `email`, `password`, `role`.
Passo 6: `providerId` errado	`providerId` lido do corpo da requisição, e não do token.
Passos 13–15 vermelhos com o status certo	Erro sem `application/problem+json`, ou com `type` diferente do Mapa.

8. O checkpoint final no ServiceHub

↑ conteúdo

A seção 12 do Mapa de Construção é a lista de critérios de aceite. A banca confere item a item; a sua equipe deve conferir antes. Os critérios se dividem em quatro grupos. O grupo Processo é o mais deixado para a véspera, e o que menos se conserta nela: o histórico do Git e os ADRs não se fabricam de forma convincente na noite anterior.

Grupo	Critérios
Funcionamento	`docker compose up -d --build` sobe API e banco sem passo manual; API em `localhost:8090`; CORS para 5173; dados persistem após `down` e `up`; fluxo completo; rotas da seção 6 com os status indicados; erros em ProblemDetail e nenhum 500 para erro do cliente.

Grupo	Critérios
Banco	Schema igual ao da seção 3, alcançado por migrações Flyway. Nenhuma migração antiga editada.
Qualidade	Ao menos um teste por regra RN01–RN13; integração com PostgreSQL real; teste de 401 nas rotas protegidas; GitHub Actions verde no último commit; Swagger em `/docs` coerente.
Processo	`docs/diagnostico.md`; commits pequenos e claros; README com como rodar, testar e variáveis; `.env.example` versionado e nenhum segredo; ao menos três decisões registradas.

Funcionamento se prova com comandos

Num clone limpo, de preferência na máquina de outra pessoa da equipe:

```
docker compose up -d --build
docker compose ps                          # api e postgres healthy
curl http://localhost:8090/actuator/health # "UP"; /api/v1/info: "OK"
docker compose down                        # SEM -v; depois:
docker compose up -d                       # o usuário cadastrado continua lá?
```

A persistência depende do volume nomeado `pg_data`, declarado no `docker-compose.yml` para o serviço `postgres`; a API só sobe depois que o banco responde ao healthcheck (`depends_on` com `condition: service_healthy`) e é publicada em `${API_PORT:-8090}:8080`.

ARMADILHA CLÁSSICA

`docker compose down -v` apaga o volume e, com ele, os dados do banco. Quem roda esse comando na demonstração conclui, errado, que a persistência não funciona.

CORS e as rotas protegidas

Sem CORS para a origem da bancada, o navegador bloqueia as chamadas antes de chegarem à API. O Mapa exige origens explícitas vindas de variável de ambiente, nunca `*`; no projeto de referência, o `SecurityConfig` lê `servicehub.cors.allowed-origins`, preenchida por `CORS_ALLOWED_ORIGINS`. Já o critério "401 nas rotas protegidas" é coberto por `ProtectedRoutesIntegrationTest`, um teste parametrizado com `@CsvSource`: cada linha é uma rota protegida, e o método `deveRetornar401QuandoSemToken` confere, para todas, o 401 e o `type` `urn:servicehub:problem:unauthorized`.

Checklist das últimas 24 horas

Técnico

- ☐ Clone limpo sobe com `docker compose`
- ☐ `mvnw verify` verde na sua máquina
- ☐ GitHub Actions verde no último commit
- ☐ Bancada com os 15 passos verdes
- ☐ Swagger em `/docs` coerente com a API
- ☐ Nenhum 500 no log depois da bancada

Entrega

- ☐ Repositório acessível ao professor
- ☐ README: rodar, testar, variáveis
- ☐ `.env.example` no Git, `.env` fora
- ☐ `docs/diagnostico.md` e três ADRs
- ☐ Tag `etapa-final` no repositório
- ☐ Roteiro ensaiado do início ao fim

A tag fecha a série `etapa-N` do semestre e marca exatamente a versão que a banca vai clonar:

```
git tag -a etapa-final -m "versão da banca"
git push origin etapa-final
```

9. A banca: roteiro e perguntas

↑ conteúdo

A banca vê a API funcionando e depois pergunta por quê. Nas palavras do Mapa: **você é avaliado pelas decisões, não pela digitação**. Um roteiro em cinco movimentos:

1. **Problema:** que dor o ServiceHub resolve, os papéis e a jornada da ordem.
2. **Arquitetura:** o diagrama de container (seção 4), as camadas e o Compose que sobe tudo.
3. **Bancada:** os 15 passos ao vivo, sem cortes.
4. **Decisões:** três ADRs, com problema, decisão e consequência, inclusive o custo.
5. **Prova:** quebrar uma regra no código e ver o teste falhar.

A bancada vem cedo: o resto da conversa parte do que a banca acabou de ver. O tempo de cada equipe é definido no AVA; se faltar, encurte problema e arquitetura, nunca a bancada e a prova.

A prova: quebrar uma regra diante da banca

A RN09 diz que só se avalia uma ordem concluída. No `ReviewService` do projeto (resumido):

```
public ReviewResponseDTO create(Long requestId, ReviewCreateDTO dto) {
    ...
    if (request.getStatus() != RequestStatus.COMPLETED) {
        throw new BusinessRuleException(
            "Só é possível avaliar ordens concluídas (status atual: "
            + request.getStatus() + ").");
    }
    ...
}
```

O teste que a protege fica em `ReviewIntegrationTest`:

```
@Test
@DisplayName("RN09: só depois de COMPLETED")
void deveRetornar409QuandoAvaliarOrdemNaoConcluida() {
    ResponseEntity<Map> resp = avaliar(maria, ordem(), 5);
    assertThat(resp.getStatusCode()).isEqualTo(HttpStatus.CONFLICT);
}
```

Ao vivo: mostre a regra e o teste; comente o `if`; rode `mvnw.cmd test -Dtest=ReviewIntegrationTest` e veja o teste ficar vermelho, porque o 409 virou 201; desfaça a mudança, rode de novo e veja o verde. O teste usa Testcontainers, então o Docker precisa estar rodando. Cada equipe escolhe a própria regra e ensaia a quebra antes.

Plano B. Se a demonstração falhar, faça o **diagnóstico em voz alta**: leia o ProblemDetail, pegue o `X-Request-Id` e ache a linha no log; investigar também é demonstrar. Relatório do Playwright da véspera, vídeo do ciclo e coleção do Postman provam que funcionou antes, mas não substituem a demonstração.

Perguntas típicas e onde está a resposta

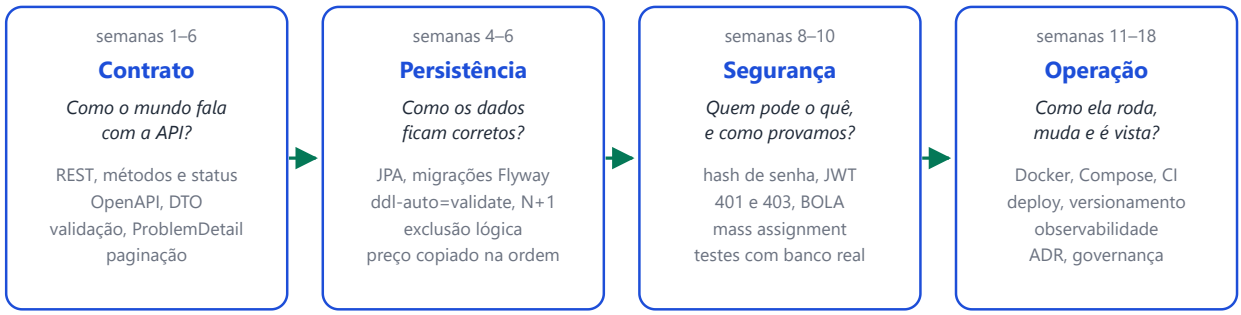
Não decore a coluna da direita: a banca pede o arquivo e a linha do *seu* código.

Pergunta	Linha de resposta
Por que 409 e não 400 aqui?	400 é requisição mal formada. 409 é requisição válida que viola uma regra ou o estado atual: e-mail repetido, transição proibida, avaliação duplicada.
Onde a API impede um cliente de ver a ordem de outro?	No service, comparando o usuário do token com os participantes da ordem: 403 (RN13, BOLA).
Por que o `providerId` não vem no corpo?	Mass assignment: quem publica é quem está no token. Campos que o cliente não pode definir não existem no DTO.
E o token de um usuário desativado?	Deixa de valer: o `JwtUserConverter` carrega o usuário pelo id do token a cada requisição e, inativo, responde 401 (RN11).
Por que exclusão lógica?	Apagar um prestador apagaria ordens e avaliações de outras pessoas. Excluir é marcar `is_active = false`; as FKs são RESTRICT.
Por que a ordem guarda `totalPrice`?	É o preço no momento da contratação. Se o prestador reajustar o serviço, a ordem já feita não muda.
Esta parte foi a IA que escreveu?	Responda com honestidade e evidência: o teste que prova a regra e o commit em que a equipe revisou o código.

10. Síntese do semestre e revisão para a 3ª VA

↑ conteúdo

O ServiceHub foi construído em quatro etapas, cada uma respondendo a uma pergunta.



Mapa conceitual do semestre. Os conceitos se cruzam: o 403 de BOLA depende do contrato de erro, do dono gravado no banco e do token.

Pontos de revisão para a 3ª VA

Tema	Saiba explicar	Pergunta para treinar
Contrato	Status HTTP, idempotência, DTO, ProblemDetail, versão na URL	Por que 409 e não 400 numa transição proibida?
Persistência	Migração versionada, `validate`, N+1, exclusão lógica	Por que nunca editar uma migração já aplicada?
Segurança	Hash, JWT, autenticação × autorização, BOLA, mass assignment	Onde se confere que a ordem é do usuário?
Testes	Unidade × integração, Testcontainers, cobertura	Por que testar com PostgreSQL real e não H2?
Operação	Container, Compose, CI, pilares da observabilidade, SLO × SLA, ADR, depreciação	Que dado você usaria para decidir desligar a v1?

COMO RESPONDER

Responda com **conceito, exemplo e consequência**, e treine exemplos fora do ServiceHub (o formato da 3ª VA sai no AVA). Ex.: "Idempotência: repetir a operação deixa o servidor no mesmo estado (conceito). Dois `PUT /services/12` iguais produzem o mesmo serviço (exemplo). Logo, o cliente pode repetir um PUT após falha de rede sem duplicar dados (consequência)."

11. Construindo com IA

↑ conteúdo

Na reta final, a IA não escreve funcionalidades: ajuda a **auditar** e a **treinar**. **Especificar:** a seção 12 do Mapa é a especificação. **Gerar:** relatório critério por critério, com arquivo de evidência, sem alterar código. **Revisar:** cada "atendido" vira um comando que você roda. **Provar:** bancada verde, `mvnw verify` verde e uma regra quebrada de propósito.

Prompt fraco e prompt bom

Fraco

"Meu projeto está pronto para a banca? Se faltar algo, conserta." Pede opinião e autoriza mexer no código na véspera.

Bom

Entrega a seção 12, proíbe alterações e exige, por item, arquivo e comando de verificação.

O prompt desta semana

Dois pedidos, em conversas separadas: auditar e treinar a defesa.

```
Contexto: equipe de Engenharia de Software terminando a API ServiceHub
(Java 21, Spring Boot 4.1, Spring Security 7 + JWT HS256, PostgreSQL 16,
Flyway, Testcontainers 2, Docker Compose, GitHub Actions). A banca confere
os critérios abaixo, item a item, contra a tag etapa-final.
<cole aqui a seção 12 do Mapa: Funcionamento, Banco, Qualidade, Processo>

Objetivo: auditar este repositório contra CADA critério acima.

Restrições:
- NÃO altere, crie nem apague arquivos: auditoria só de leitura.
- NÃO rode docker compose down -v; NÃO leia o .env (só o .env.example).
- Só marque "atendido" depois de abrir o arquivo e citar o trecho.
- Para RN01–RN13, nomeie o teste de cada regra; sem teste = "não encontrado".
- Para "nenhuma migração antiga editada", use o git log das migrações.
- Não sugira correções agora; só o diagnóstico.

Antes do relatório, mostre o plano: arquivos que vai inspecionar e
comandos que EU devo rodar para confirmar cada grupo. Espere meu ok.

Saída: tabela critério | atendido/parcial/ausente | evidência (arquivo
e trecho) | comando de confirmação.
Critério de aceite: todos os itens da seção 12 na tabela, nenhum
"atendido" sem arquivo e comando, git status limpo ao final.
```

```
Contexto: vou defender a API ServiceHub numa banca. Seguem os ADRs da
equipe; o contrato é o Mapa de Construção.
<cole aqui os ADRs da equipe>

Objetivo: me treinar para a banca. Você é o avaliador, não o autor.

Restrições:
- NÃO escreva respostas modelo para eu decorar.
- UMA pergunta por vez: "por que X e não Y?", "onde no código isso
  está?", "qual o custo dessa decisão?".
- Após minha resposta, diga qual parte faltou (problema, alternativas,
  critério, custo, evidência) e faça uma pergunta de acompanhamento.
- "Porque é a melhor prática" não vale: cobre o critério.
- Não invente fatos sobre o meu código; pergunte.

Antes de começar, liste os temas que vai cobrir (contrato, erros,
segurança, BOLA, domínio, banco, testes, operação) e espere meu ok.
Critério de aceite: após 8 perguntas, respondo cada uma citando um
arquivo do meu projeto e o teste que prova a resposta.
```

O que a IA costuma errar nesta etapa

- Declarar "tudo pronto" sem ter rodado nada. Peça a evidência, não a opinião.
- Sugerir `docker compose down -v` para "limpar": o `-v` apaga os dados do banco.
- "Consertar" um teste vermelho trocando o 409 esperado pelo 201 recebido, em vez de corrigir o código.
- Editar uma migração já aplicada em vez de criar a próxima: o Flyway recusa o checksum alterado.
- Liberar CORS com `*`; o Mapa pede origens explícitas por variável de ambiente.

Checklist de revisão

☐ Cada "atendido" tem um arquivo que eu abri e um comando que eu rodei?

☐ Cada regra RN01–RN13 tem um teste nomeado, e eu li o teste?

☐ `git status` está limpo depois da auditoria?

☐ Algum diff tocou em `src/test`? Revisei cada asserção alterada?

☐ Respondo às perguntas da banca mostrando arquivo e linha, sem texto pronto?

Prova

Num clone limpo da tag `etapa-final`: `docker compose up -d --build`, `npm run teste` com os 15 passos verdes, `mvnw verify` verde e uma regra quebrada de propósito deixando o teste certo vermelho. Depois, cada pessoa responde a uma pergunta da seção 9 com o arquivo do próprio projeto.

12. Glossário

↑ conteúdo

Defesa técnica	Explicação de uma escolha pelo problema, alternativas, critério, custo e evidência.
Trade-off	Troca em que se ganha uma característica abrindo mão de outra.
ADR	Registro de decisão de arquitetura: contexto, decisão e consequências.
Modelo C4	Forma de diagramar software em quatro níveis de zoom: contexto (o sistema e quem o usa), container, componente e código.
Container (C4)	Parte que roda e guarda estado separadamente, como uma API ou um banco. Não é o mesmo que container Docker.
Dívida técnica	Custo futuro de um atalho tomado hoje; os juros são o encarecimento de cada mudança.
Limitação declarada	Algo que o sistema conscientemente não faz, registrado com motivo, impacto e caminho.
Rubrica	Tabela de critérios com níveis descritos, usada para avaliar de forma consistente.
Bancada de testes	Ferramenta que roda o mesmo ciclo de chamadas contra cada API e confere o contrato.
Rate limiting	Limite de requisições por cliente num intervalo; o excesso recebe 429.
Tracing distribuído	Rastreamento de uma requisição através de vários serviços, por identificadores de trace e span.
Tag anotada	Marca com nome, autor e mensagem num commit do Git, como `etapa-final`.

13. Exercícios

↑ conteúdo

Os exercícios 1 a 6 são perguntas típicas de banca: responda como responderia em voz alta, citando o arquivo do seu projeto. Os demais revisam o semestre.

1. Transforme esta descrição numa defesa técnica com as cinco partes: "Usamos Flyway para o banco".
2. Por que a API responde 409, e não 400, quando o cliente tenta avaliar uma ordem que ainda está IN_PROGRESS? E quando ele envia a nota 7?
3. Onde, no seu projeto, a API impede que um cliente veja a ordem de outro? Que status volta, e por que esse status e não 404?
4. Um avaliador pergunta: "por que vocês não usaram H2, que é mais rápido?". Responda com contexto, critério, custo e evidência.
5. Declare, no formato de quatro colunas, uma limitação do seu projeto que não esteja na tabela da seção 5.

6. A bancada mostra o passo 6 vermelho com a mensagem "providerId deveria ser o do prestador autenticado (vem do token)". Qual é a causa provável, que vulnerabilidade ela representa e por que os passos 7 a 12 aparecem como pulados?
7. Desenhe o diagrama de contexto (nível 1 do C4) do ServiceHub. Quais pessoas e sistemas aparecem? O pgAdmin entra? Justifique.
8. Classifique no quadrante de Fowler: (a) a equipe aceitou, sem ler, um controller gerado por IA que acessa o repositório direto; (b) a equipe decidiu não paginar uma rota interna e registrou isso num ADR, com o plano de paginar se a lista passar de cem itens.
9. Um colega roda `docker compose down -v`, sobe de novo e diz que "a persistência não funciona". Explique o que aconteceu e qual critério da seção 12 ele está tentando verificar.
10. Por que nunca se edita uma migração Flyway já aplicada? O que acontece na próxima subida da aplicação, e qual é o jeito certo de mudar o schema?
11. Explique a diferença entre autenticação e autorização usando dois passos da bancada como exemplo.
12. Durante a auditoria com IA, o assistente propõe "corrigir" um teste vermelho trocando `HttpStatus.CONFLICT` por `HttpStatus.CREATED`. Explique por que isso é errado e escreva a instrução que você acrescentaria ao prompt para impedir esse tipo de mudança.

14. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RICHARDS, M.; FORD, N. *Fundamentos da arquitetura de software*. Alta Books, 2024. Capítulo 21, sobre diagramar e apresentar a arquitetura.
- MARTIN, R. C. *Desenvolvimento ágil limpo*. Alta Books, 2020. Capítulo 7.
- Revisão: RICHARDS, M.; FORD, N. *Fundamentos da arquitetura de software*. Capítulos 4 a 6 e 19, sobre características e decisões de arquitetura.
- Para a 3ª VA: as leituras indicadas ao longo do semestre e o Mapa de Construção, guiados pelo mapa conceitual da seção 10.

Referências abertas:

- NYGARD, M. *Documenting Architecture Decisions*. 2011.
- IETF. RFC 9457: *Problem Details for HTTP APIs*. 2023.
- OWASP. *API Security Top 10*. 2023.
- Documentação oficial do Spring Boot: `docs.spring.io/spring-boot`.
- Documentação oficial do Playwright: `playwright.dev`.